The Mathematics Behind Large Language Models

From Tokens to Tensors

Coert van Gemeren, Ph.D.

University of Applied Sciences Utrecht

July 16, 2026

Agenda

1. Tokens
2. Vectors and embeddings
3. Transformers and attention

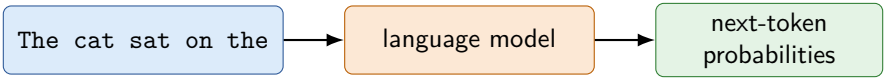
CHAPTER 1

Tokens

From text to the discrete units a model can process

An LLM is a next-token predictor

A generative model learns a probability distribution over what could come next. For a language model, the possible outputs are tokens from vocabulary V.



Candidate	Probability
mat	0.46
floor	0.21
sofa	0.12
other tokens	0.21

1. Choose or sample one next token.
2. Append it to the existing sequence.
3. Feed the longer sequence back into the model.
4. Repeat until generation stops.

Why begin with tokens?

The model generates one token at a time, so we first need to understand what tokens are and how text becomes a token sequence.

A token is a discrete piece of data

Text

`ing` `,`

subword or punctuation

Image

patch or visual code

Audio

short segment or codec code

Common idea

A model processes a sequence of numbered units. The tokenizer decides where one unit ends and the next begins.

Text can be split in different ways

Consider the same input: `unbelievable!`

Characters	`u` `n` `b` `e` `l` `i` `e` `v` `a` `b` `l` `e` `!`
Words	`unbelievable` `!`
Subwords	`un` `believ` `able` `!`

Why subwords?

Frequent strings can become one token, while rare words can still be assembled from smaller pieces. A token is therefore not necessarily a character or a complete word.

The subword split above is illustrative; the result depends on the tokenizer and its training data.

How repeated strings become tokens: toy BPE

Byte Pair Encoding (BPE) starts with characters; each word in this toy corpus occurs once.

`play` `played` `player` `playing`

At every round, count *adjacent pairs across the whole corpus*, then replace one most-frequent pair with a new symbol.

Round	Selected pair	Count	Common prefix
0	–	–	`p l a y`
1	`p + l`	4	`pl a y`
2	`pl + a`	4	`pla y`
3	`pla + y`	4	`play`

Initially several pairs tie at count 4; a fixed tie-breaking rule chooses which equally frequent pair is merged first.

What does count 4 mean?

The selected pair appears once in each of the four words, for four occurrences across the corpus.

After round 3:

`played` → `[play][e][d]`
`player` → `[play][e][r]`

Stopping criterion

This example stops after round 3 only to highlight `play`. Real BPE continues to a merge budget or target vocabulary size. Here, the next merge is `play + e`, which occurs twice.

A vocabulary maps tokens to integer IDs

Tiny illustrative vocabulary

ID		Token
7	↔	`!`
41	↔	`<space>`
88	↔	`er`
314	↔	`play`

Input begins with a space: `player!`

`<space>` `play` `er` `!`

$\longrightarrow [41, 314, 88, 7]$

IDs are labels, not quantities

ID 314 is not "larger in meaning" than ID 88. The numbers only index entries in a fixed list.

Whitespace and punctuation also carry information. Depending on the tokenizer, a space may be separate or attached to a neighboring token.

The complete collection of all available tokens is the vocabulary, denoted V.

CHAPTER 2

Vectors and embeddings

From token IDs to learned numerical representations

Scalar, vector, matrix, tensor

These names describe how numbers are arranged into axes:

Name	Example	Rank and shape
Scalar	$s = 3.2$	rank 0, shape $()$
Vector	$\mathbf{x} = \begin{bmatrix} 0.2 & -0.7 & 0.4 \end{bmatrix}$	rank 1, shape (3)
Matrix	$X = \begin{bmatrix} 0.2 & -0.7 & 0.4 \\ 0.1 & 0.8 & -0.3 \end{bmatrix}$	rank 2, shape $(2, 3)$
Tensor	$T = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \end{bmatrix}$	rank 3, shape (b, n, d)

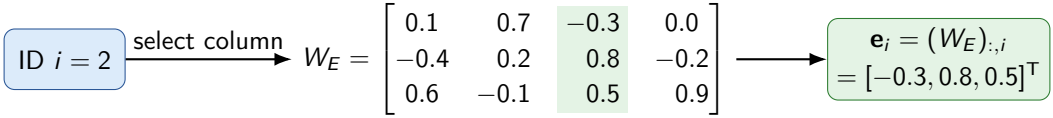
Terminology

A tensor is the general object; a scalar, vector, and matrix are rank-0, rank-1, and rank-2 tensors. Shape gives the size along every axis.

Embedding lookup: from an ID to a vector

Let $|V|$ be the number of tokens. The embedding dimension d is a model hyperparameter: the chosen width of every token vector. The model stores the learnable *embedding matrix* W_E:

$$W_E \in \mathbb{R}^{d \times |V|}.$$

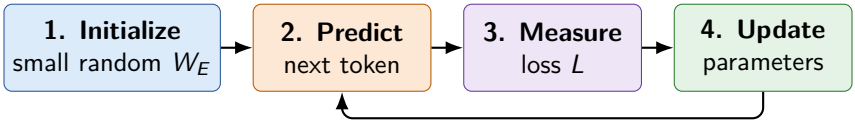


Lookup versus learning

During a forward pass, lookup simply selects a column. During training, gradients update the values in every token column. Over many examples, the token vectors develop meaningful geometric relationships based on how the tokens are used.

Training learns the embedding values

The embedding matrix W_E is a set of model parameters. Token IDs select its columns, but next-token training determines the numbers stored in them.



Next-token loss

If the correct next token is y, the cross-entropy contribution is

$$L = -\log p(y \mid x).$$

High probability for y gives a small loss; low probability gives a large loss.

Learning from the loss

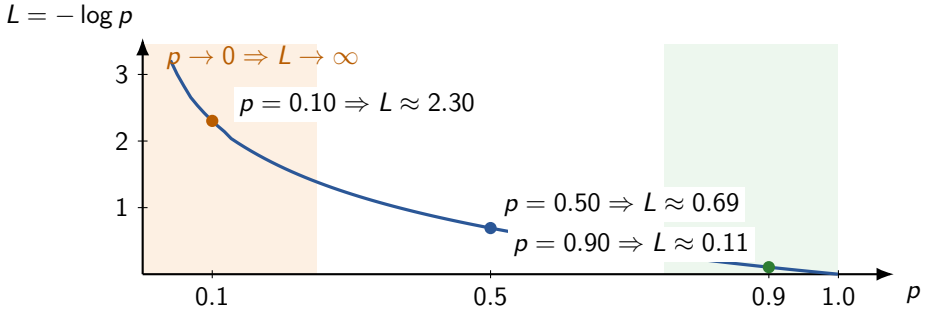
Backpropagation computes $\nabla_{W_E} L$. An optimizer with learning rate η updates the matrix:

$$W_E \leftarrow W_E - \eta \nabla_{W_E} L.$$

Repeating this loop gives the columns useful values.

Cross-entropy strongly penalizes low probability

Let $p = p(y \mid x)$ be the probability assigned to the correct next token. Its loss is $L(p) = -\log p$ (using the natural logarithm).

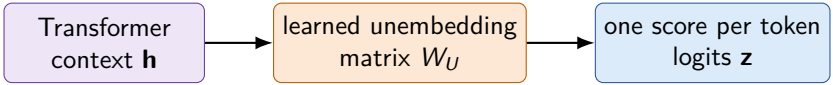


Training signal

A confident wrong prediction receives a large penalty; assigning the correct token probability near 1 drives its loss toward 0.

Unembedding is needed to make predictions

To evaluate the loss L, the model must make a next-token prediction. For now, treat the Transformer as a black box: it turns the input token vectors into a context vector $\mathbf{h}$.



Embedding W_E

Token ID $\longrightarrow$ input token vector

Unembedding W_U

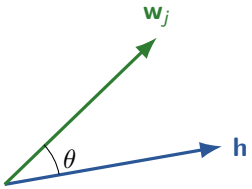
Context vector $\longrightarrow$ token scores

Why training needs both matrices

W_E supplies the vectors that enter the Transformer. W_U supplies one candidate vector for every possible next token, letting the model produce the scores needed for its prediction and loss.

Dot product: scoring a candidate token

Input token vectors come from columns of W_E. After the Transformer has combined their context, candidate token j uses column $\mathbf{w}_j = (W_U)_{:,j}$. Its logit is the dot product $z_j = \mathbf{h}^\mathsf{T}\mathbf{w}_j$.



$$\mathbf{h}^\mathsf{T}\mathbf{w}_j = \|\mathbf{h}\|\|\mathbf{w}_j\| \cos\theta$$

Dot-product intuition

The score grows when the vectors point in similar directions, but also when either vector is longer.

Cosine removes length

$$\cos\theta = \frac{\mathbf{h}^\mathsf{T}\mathbf{w}_j}{\|\mathbf{h}\|\|\mathbf{w}_j\|}.$$

Dividing out both lengths keeps only directional similarity.

Connection to cross-entropy

Dot products produce candidate scores. Cross-entropy pushes the correct token's score above its competing tokens.

From logits to probabilities: softmax

The model produces one raw score, or *logit* z_j, for every token $j \in V$. Logits can be any real numbers and do not have to sum to 1.

Normalize all vocabulary scores together

$$p_j = p(y = j \mid x) = \frac{e^{z_j}}{\sum_{k \in V} e^{z_k}}$$

- $e^{z_j} > 0$ makes every result positive.
- The shared denominator makes $\sum_{j \in V} p_j = 1$.
- Larger logits receive larger probabilities.

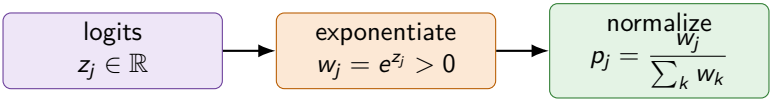
Candidate token	Logit z_j	e^{z_j}	Probability p_j
`cat`	2.0	7.39	0.67
`dog`	1.0	2.72	0.24
`runs`	0.0	1.00	0.09
		11.11	1.00

Connecting probabilities to the loss

If the correct token is `cat`, then $p_y = 0.67$ and $L = -\log(0.67) \approx 0.40$.

Why does e appear in softmax?

"That's a great question, because the answer isn't 'because mathematicians like e'".



Why an exponential?

It is positive, preserves order, and a common shift cancels:

$$\frac{e^{z_j+c}}{\sum_k e^{z_k+c}} = \frac{e^c e^{z_j}}{e^c \sum_k e^{z_k}} = \frac{e^{z_j}}{\sum_k e^{z_k}}.$$

Only differences between logits affect the probabilities.

Why specifically base e?

Any base $a > 1$ could work, since

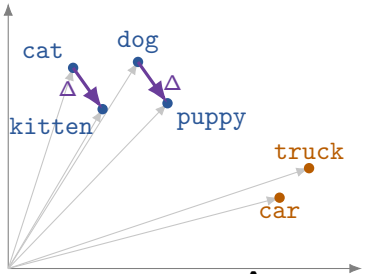
$$a^z = e^{(\ln a)z}.$$

The base only rescales the logits. But

$$\frac{d}{dz}a^z = (\ln a)a^z, \qquad \frac{d}{dz}e^z = e^z.$$

Base e removes the extra factor, keeping learning gradients clean.

Embeddings turn usage into geometry



$$\mathbf{e}_{\text{puppy}} - \mathbf{e}_{\text{dog}} \approx \boldsymbol{\Delta},$$
$$\mathbf{e}_{\text{kitten}} - \mathbf{e}_{\text{cat}} \approx \boldsymbol{\Delta}.$$

Both arrows show the same relative move: adult animal $\rightarrow$ young animal.

Tokens used in similar contexts often acquire related locations.

Cosine similarity measures directional similarity:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}.$$

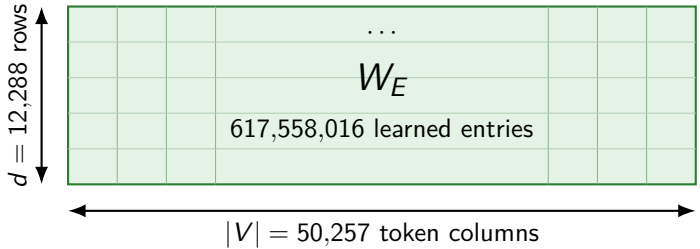
Measure, not objective

Next-token cross-entropy is the training objective; cosine similarity only inspects learned geometry. Lookup embeddings are static; contextual representations come later.

Scale example: the GPT-3 175B embedding matrix

The largest GPT-3 model uses a vocabulary of 50,257 tokens and an embedding dimension of $d = 12{,}288$.

$W_E \in \mathbb{R}^{12{,}288 \times 50{,}257}$ (d rows, one column per token).



Parameter count and matching dimensions

$$50{,}257 \times 12{,}288 = \boxed{617{,}558{,}016} \approx 618 \text{ million parameters.}$$

For GPT-3, W_U has the same shape as W_E: $W_U, W_E \in \mathbb{R}^{12{,}288 \times 50{,}257}$. Equal shape does not mean equal entries.

A token sequence becomes a matrix

`the` `cat` `sleeps` $\longrightarrow$ $[12, 51, 908]$

Each ID selects one column from W_E. Stacking the transposed vectors gives

$$X = \begin{bmatrix} \mathbf{e}_{12}^{\mathsf{T}} \\ \mathbf{e}_{51}^{\mathsf{T}} \\ \mathbf{e}_{908}^{\mathsf{T}} \end{bmatrix} \in \mathbb{R}^{n \times d}, \qquad n = 3.$$

one sequence: $n \times d$ —— stack b sequences ——▶ batch: $T \in \mathbb{R}^{b \times n \times d}$

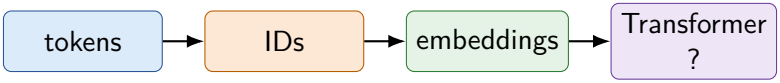
Sequences in a batch are commonly padded to a shared length; a mask tells later layers which positions are padding.

CHAPTER 3

Transformers and attention

From static vectors to context-aware representations

From token vectors to contextual vectors



The embedding lookup gives `bank` the same starting vector in both sentences:

- "They sat on the river **bank**".
- "She deposited money at the **bank**".

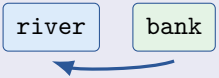
The problem self-attention solves

How can every token exchange information with the other tokens so that its vector reflects the surrounding context? The following slides build the mathematical mechanism that makes this possible.

Attention creates a context-dependent update

The lookup vector for `bank` starts identically in both sentences. Attention lets it collect different clues from the surrounding tokens; we denote the resulting contextual vector by $\mathbf{h}$.

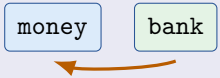
River context



$$\mathbf{e}_{\text{bank}} \longrightarrow \mathbf{h}_{\text{bank}}^{(\text{river})}$$

The result incorporates information associated with a river edge.

Financial context



$$\mathbf{e}_{\text{bank}} \longrightarrow \mathbf{h}_{\text{bank}}^{(\text{money})}$$

The result incorporates information associated with finance.

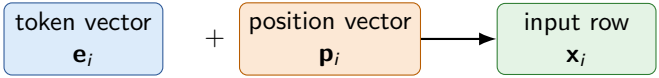
The central idea

Each position asks which other positions matter, then mixes information from them. The resulting vector is contextual rather than a static lookup.

Before attention: the model needs token order

A sentence is not merely a bag of tokens: changing the order changes the meaning. A basic construction adds a position vector to every token vector:

$$\mathbf{x}_i = \mathbf{e}_{\text{token}_i} + \mathbf{p}_i, \qquad X \in \mathbb{R}^{n \times d}.$$



Implementations differ

Position information may be learned or fixed. Rotary position encodings instead inject relative position when queries and keys are compared. The shared goal is the same: make order available to attention.

Query, key, value: three roles for each token

Self-attention derives three vectors from every input position.

Query $\mathbf{q}_i$
What information do I need?

Key $\mathbf{k}_j$
What kind of information do I offer?

Value $\mathbf{v}_j$
What information will I contribute?

For position i:

1. compare its query $\mathbf{q}_i$ with every allowed key $\mathbf{k}_j$;
2. turn the comparison scores into attention weights;
3. use those weights to combine the value vectors $\mathbf{v}_j$.

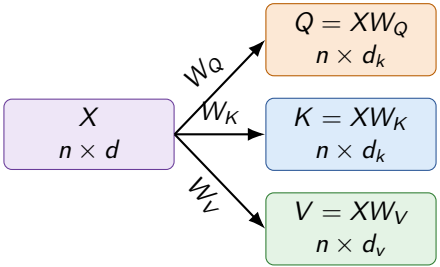
Why "self"-attention?

Queries, keys, and values all come from the same sequence X.

Queries, keys, and values are learned projections

Three learned matrices give each input row three different roles:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V.$$



Shapes

Weights: $W_Q, W_K \in \mathbb{R}^{d \times d_k}$, $W_V \in \mathbb{R}^{d \times d_v}$.
Outputs: $Q, K \in \mathbb{R}^{n \times d_k}$, $V \in \mathbb{R}^{n \times d_v}$.

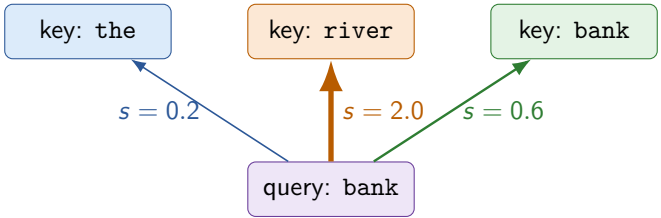
How are they learned?

There are no separate query, key, or value targets. Next-token cross-entropy backpropagates through attention and updates W_Q, W_K, W_V with all other model weights.

Step 1: compare one query with every key

Suppose position i contains `bank`. Its compatibility with an allowed position j is the scaled dot product

$$s_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d_k}}.$$



Interpretation

A larger score means the query and key are more compatible. A score is still an unrestricted number, not a probability.

Why divide by $\sqrt{d_k}$?

Dot products tend to grow as vectors get wider. Scaling keeps scores in a range where softmax remains responsive.

A causal mask prevents looking into the future

During next-token prediction, position i may use only positions $j \le i$. Forbidden scores are replaced by $-\infty$ before softmax.

$$S = \frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M, \qquad M_{ij} = \begin{cases} 0 & j \le i, \\ -\infty & j > i. \end{cases}$$

Because $e^{-\infty} = 0$, a masked position receives zero attention weight.

Rows attend to columns

	the	river	bank	rose
the	s_{11}	$-\infty$	$-\infty$	$-\infty$
river	s_{21}	s_{22}	$-\infty$	$-\infty$
bank	s_{31}	s_{32}	s_{33}	$-\infty$
rose	s_{41}	s_{42}	s_{43}	s_{44}

No information leakage

The representation at position i uses tokens $1, \dots, i$ to predict token $i+1$. It cannot depend on token $i+1$ or anything after it.

Step 2: softmax turns scores into attention weights

For a fixed query position i, normalize its allowed scores across j:

$$\alpha_{ij} = \frac{e^{s_{ij}}}{\sum_{\ell \le i} e^{s_{i\ell}}}, \qquad \alpha_{ij} > 0, \qquad \sum_{j \le i} \alpha_{ij} = 1.$$

Key token	Score s_{ij}	$e^{s_{ij}}$	Weight α_{ij}
`the`	0.2	1.22	0.12
`river`	2.0	7.39	0.71
`bank`	0.6	1.82	0.17
		10.43	1.00

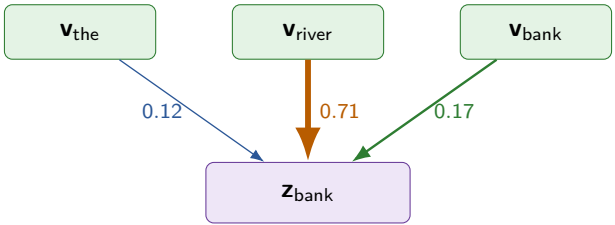
What the weights say

For this illustrative head, `bank` assigns most of its attention to `river`. Softmax makes the comparison relative: all allowed tokens compete for a fixed total weight of 1.

Step 3: combine values into contextual information

The weights decide how much of every value vector reaches position i:

$$\mathbf{z}_i = \sum_{j \le i} \alpha_{ij} \mathbf{v}_j.$$



$$\mathbf{z}_{\text{bank}} = 0.12\mathbf{v}_{\text{the}} + 0.71\mathbf{v}_{\text{river}} + 0.17\mathbf{v}_{\text{bank}}.$$

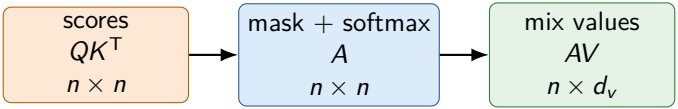
The context has entered the vector

The update for `bank` now contains substantial information from `river`. Different contexts yield different weights and therefore different updates.

All positions at once: the attention equation

The per-token operations combine into one matrix expression:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\mathsf{T}}}{\sqrt{d_k}} + M\right) V$$

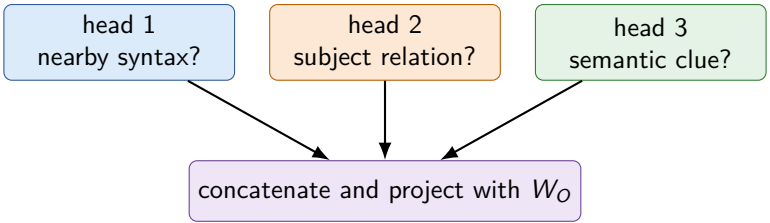


Reading the shape

The $n \times n$ matrix A contains one attention distribution per row. Multiplying by V produces one contextual update for every position.

Multi-head attention learns several relationships

One attention pattern need not capture every useful relationship. A Transformer runs H attention heads with separate learned projections.

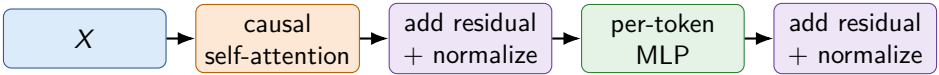


$$\text{MultiHead}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)W_O.$$

Interpret the labels as intuition

Heads are not assigned these jobs by hand; training learns their behavior, and real heads may mix several kinds of relationships.

Attention is the communication step in a Transformer block



Across positions

Self-attention moves information between tokens. The causal mask preserves next-token prediction.

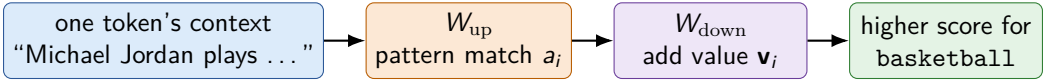
Within each position

The MLP transforms each token vector independently. Residual paths help preserve and refine existing information.

Depth builds richer context

Stacking many blocks lets information travel and be transformed repeatedly, producing the contextual representations used to predict the next token.

Inside the per-token MLP: factual associations



$$\text{MLP}(\mathbf{x}) = \sigma(\mathbf{x}W_{\text{up}})W_{\text{down}} = \sum_i a_i \mathbf{v}_i, \qquad a_i = \sigma(\mathbf{x}\mathbf{k}_i).$$

$\mathbf{k}_i$ is column i of W_{up}; $\mathbf{v}_i$ is row i of W_{down}.

Geva et al. (2021)

Transformer Feed-Forward Layers Are Key-Value Memories. They found that W_{up} directions respond to interpretable textual patterns, while corresponding W_{down} values favor plausible output tokens.

Meng et al. (2022): ROME

Locating and Editing Factual Associations in GPT. Causal tracing localized factual predictions to middle-layer MLPs; rank-one changes to their feed-forward weights could update particular factual associations.

Important nuance

"Michael Jordan plays basketball" is illustrative. Evidence points to distributed, compositional storage—not one complete sentence in one neuron.

GPT-3: sizes of the attention projection matrices

GPT-3 uses $H = 96$ attention heads in each of $N = 96$ layers, with

$$d_q = d_k = d_v = 128, \qquad d_{\text{embed}} = 12{,}288.$$

From separate heads to one combined projection

$Hd_q = Hd_k = Hd_v = 96 \cdot 128 = 12{,}288 = d_{\text{embed}}$. Thus each combined projection matrix is square.

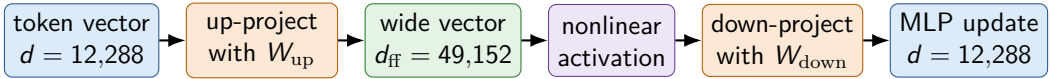
Matrix	Shape in one layer	Parameters per layer	Across 96 layers
W_Q	$12{,}288 \times 12{,}288$	150,994,944	14,495,514,624
W_K	$12{,}288 \times 12{,}288$	150,994,944	14,495,514,624
W_V	$12{,}288 \times 12{,}288$	150,994,944	14,495,514,624
W_O	$12{,}288 \times 12{,}288$	150,994,944	14,495,514,624

Attention projections across the whole model

$4 \times 14{,}495{,}514{,}624 = 57{,}982{,}058{,}496$ learned weights, excluding biases.

GPT-3: the MLP up and down projections

Attention moves information between tokens. The **MLP** then transforms each token vector independently, using the same two matrices at every position.



Matrix	Shape in one layer	Parameters per layer	Across 96 layers
W_{up}	$12{,}288 \times 49{,}152$	603,979,776	57,982,058,496
W_{down}	$49{,}152 \times 12{,}288$	603,979,776	57,982,058,496

Most GPT-3 parameters are in its MLPs

Together: $2 \times 57{,}982{,}058{,}496 = 115{,}964{,}116{,}992$ weights—about 66% of GPT-3.

Model scale: GPT-3 175B (2020) vs. Gemma 4 31B (2026)

Both are Transformer models, but Gemma 4 reaches a much smaller parameter budget through a narrower, more modern architecture.

Parameter family	GPT-3 175B	Share	Gemma 4 estimate	Share
Embedding and output	1.24B	0.7%	$\approx$ 1.41B	$\approx$ 4.6%
Attention: Q, K, V, O	57.98B	33.1%	$\approx$ 6.10B	$\approx$ 19.8%
MLP: up and down projections	115.96B	66.2%	$\approx$ 13.88B	$\approx$ 45.2%
Vision encoder and other weights	$\approx$ 0B	$\approx$ 0%	$\approx$ 9.35B	$\approx$ 30.4%
Total	**175.18B**	**100%**	**30.7B**	$\approx$ 100%

About 5.7$\times$ fewer parameters

Gemma 4 uses 60 layers, a smaller embedding dimension, grouped-query and hybrid local/global attention, plus a vision encoder. Size alone is not a measure of capability; architecture, data, and training matter enormously.

Try it yourself: the `my-llm` notebook

The companion notebook turns the ideas from this presentation into a small, working Shakespeare language model.

What you will explore

- character-level and BPE tokenization;
- training and validation batches;
- a small Transformer model;
- Shakespeare-like text generation.

Get started

From the repository root, run:

`./setup.sh` (Linux/macOS), or

`.\setup.ps1` (Windows PowerShell).

Then open `my-llm.ipynb`, select the project's `.venv` kernel, and run the cells from top to bottom.

Start with the character tokenizer, then switch to BPE and compare the generated results.

Dive deeper with 3Blue1Brown

Start with the short LLM primer between Chapters 4 and 5, then continue with Chapters 5 and 6 of 3Blue1Brown's *Neural Networks* lecture series.

Start here

Large Language Models explained briefly

A concise bridge from next-token prediction and training to Transformer architecture.

Chapter 5

Transformers and LLMs

Tokens, embeddings, unembedding, softmax, and the overall architecture.

Chapter 6

Attention, step by step

A visual walkthrough of queries, keys, values, and attention weights.

▸ Watch Large Language Models explained briefly ▸ Open the full playlist

Both buttons are clickable in the PDF.

Questions?